Pràctica 1.2: Paral·lelització multiplicació de matrius disperses

Autors:

Satxa Fortuny Pimentel
Nawfal Aissaoui El Aissaoui

Computació Paral·lela i Massiva / Grau en Enginyeria Informàtica

Escola Tècnica Superior d'Enginyeria / Universitat Rovira i Virgili

Març 2026

Índex

1 Introducció i Objectius **2**

2 Anàlisi de Dependències de Dades **3**
2.1 Nuclis 1 i 2: MD × M i MD × MD → M . . . 3
2.1.1 Codi seqüencial . . . 3
2.1.2 Graf de Dependències dels Nuclis 1 i 2 . . . 4
2.1.3 Llistat i tipus de dependències dels Nuclis 1 i 2 . . . 4
2.2 Nucli 3: MD × MD → MD (Resultat Dispers) . . . 6
2.2.1 Codi seqüencial . . . 6
2.2.2 Graf de Dependències del Nucli 3 . . . 7
2.2.3 Llistat i tipus de dependències del Nucli 3 . . . 7

3 Optimitzacions Aplicades **9**
3.1 Inicialització paral·lela amb *first-touch* . . . 9
3.2 Introducció del paràmetre `BLOCK_SIZE` i blocs de càlcul . . . 10
3.3 Fase 1: Matriu dispersa × Matriu densa → Densa (`C1`) . . . 11
3.4 Fase 2: Matriu dispersa × Matriu dispersa → Densa (`C2`) . . . 12
3.5 Fase 3: Matriu dispersa × Matriu dispersa → Dispersa (`CD`) . . . 13

4 Avaluació del Rendiment **15**
4.1 Avaluació a la màquina Orca . . . 15
4.2 Avaluació a la màquina Teen . . . 15
4.3 Comparativa entre arquitectures . . . 16

5 Conclusions **16**

1 Introducció i Objectius

En aquest document es detalla el procés de paral·lelització d'un codi escrit en C que implementa la multiplicació de matrius disperses. Consta d'una anàlisi teòrica, una transformació i una avaluació del rendiment. Per dur a terme aquesta paral·lelització en una arquitectura de memòria compartida, s'ha fet ús de l'estàndard i la interfície de programació d'aplicacions OpenMP.
El programa consta de les següents càrregues de treball principals:

- **Generació i ordenació de matrius disperses:** Inicialització pseudoaleatòria de dues matrius disperses A i B de mida 8000×8000 amb una densitat d'elements de l'1% ($ND = N \times N/100 = 640.000$ elements no nuls). Les matrius es representen en format comprimit (`tmd`: fila, columna, valor) i s'ordenen mitjançant `qsort`: A per files i B per columnes. Es construeix un vector d'índexs `jBD` per accedir eficientment a les columnes de B.

- **Multiplicació de matrius (Nucli computacional):** S'avaluen tres aproximacions diferenciades:

  - **Nucli 1 – MD $\times$ M:** Matriu Dispersa per Matriu Densa. Recorre els elements no nuls d'A (AD) i per a cada un, multiplica per la fila corresponent de la matriu densa B, acumulant el resultat a $C1$.
  - **Nucli 2 – MD $\times$ MD $\rightarrow$ M:** Matriu Dispersa per Matriu Dispersa amb resultat dens. Utilitza un vector temporal `VBcol` per expandir columna a columna de B en format dens, calcula una columna sencera de $C2$, i neteja `VBcol` abans de la següent iteració.
  - **Nucli 3 – MD $\times$ MD $\rightarrow$ MD:** Matriu Dispersa per Matriu Dispersa amb resultat dispers. Similar al Nucli 2, però a més comprimeix el resultat a format dispers (`CD`) mitjançant un vector temporal `VCcol` i un comptador `neleC`.

El punt de partida per a l'avaluació d'aquesta pràctica són els temps d'execució de la versió seqüencial base compilada amb el flag d'optimització màxima `-O3`. Les mètriques inicials preses són:

- **Teen:** 16.37 segons.
- **Orca:** 48.71 segons.

Un cop definit el context, els objectius principals de la pràctica són:

- **Anàlisi teòrica:** Identificar les dependències de dades dels tres nuclis computacionals i generar els grafs de dependències pertinents per estudiar-ne el tractament abans d'aplicar directives de paral·lelització.

- **Preservació de la correcció:** Garantir que la paral·lelització del codi no n'altera els resultats (superant les validacions integrades de MD $\times$ M $\rightarrow$ M i MD $\times$ MD $\rightarrow$ MD) i evitar alhora condicions de carrera.

- **Maximització del rendiment:** Aplicar les estratègies de paral·lelització òptimes per minimitzar l'overhead i superar els llindars d'acceleració exigits. Concretament, l'objectiu és aconseguir:

  - Una mitjana harmònica de l'speedup ≥ 3.5 a la màquina Teen, forçant la creació de fils des de 2 fins a 32.
  - Una mitjana harmònica de l'speedup ≥ 5 a la màquina Orca, forçant la creació de fils des de 2 fins a 128.

2 Anàlisi de Dependències de Dades

Per poder dissenyar una estratègia de paral·lelització eficient i lliure de condicions de carrera, és imprescindible realitzar una anàlisi prèvia de les dependències de dades del codi seqüencial. S'analitzen els tres nuclis computacionals. Els Nuclis 1 i 2 s'analitzen conjuntament atès que comparteixen el mateix patró d'accés per columnes.

2.1 Nuclis 1 i 2: MD $\times$ M i MD $\times$ MD $\rightarrow$ M

2.1.1 Codi seqüencial

El primer nucli calcula el producte d'una matriu dispersa per una matriu densa. El segon nucli calcula el producte de dues matrius disperses amb resultat dens, utilitzant un vector temporal `VBcol` que s'expandeix i es neteja a cada iteració del bucle exterior.

Listing 1: Nucli 1: MD $\times$ M

```
// Nucli 1: Matriu dispersa per matriu densa
    for(i=0; i<N; i++)
        for (k=0; k<ND; k++)
S0:         C1[AD[k].i][i] += AD[k].v * B[AD[k].j][i];
```

Listing 2: Nucli 2: MD $\times$ MD $\rightarrow$ M

```
// Inicialitzacio
    for (j=0; j<N; j++)
        VBcol[j] = 0;

// Bucle principal (per columnes de C2)
    for(i=0; i<N; i++)
    {
        // Fase 1: Expansio de la columna i de B
            for (k=jBD[i]; k<jBD[i+1]; k++)
        S1:     VBcol[BD[k].i] = BD[k].v;

        // Fase 2: Calcul de la columna i de C2
            for (k=0; k<ND; k++)
        S2:     C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];

        // Fase 3: Neteja del vector temporal
            for (j=0; j<N; j++)
        S3:     VBcol[j] = 0;
    }
```

2.1.2 Graf de Dependències dels Nuclis 1 i 2

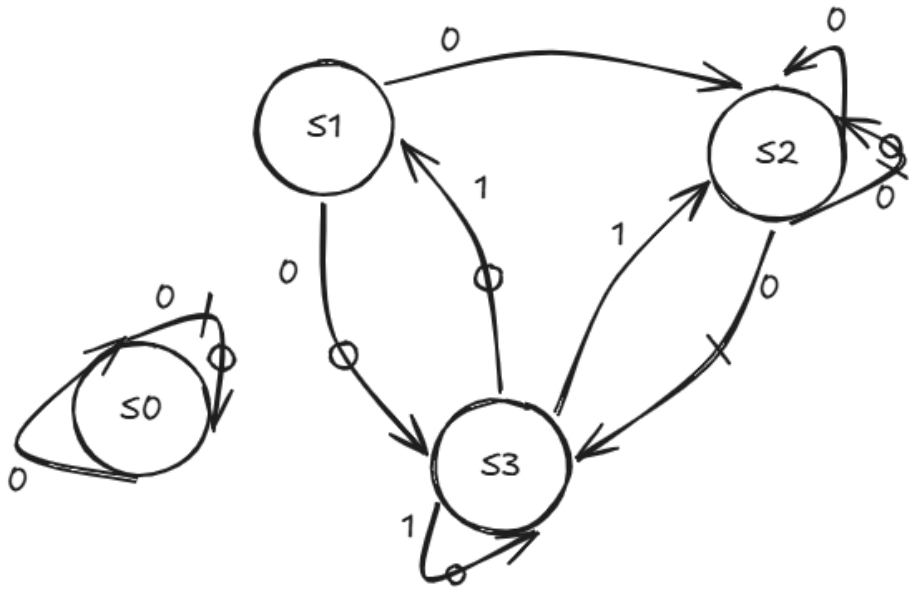

Figura 1: Graf de dependències dels Nuclis 1 i 2

2.1.3 Llistat i tipus de dependències dels Nuclis 1 i 2

S'han identificat les dependències de dades del bucle exterior `for(i=0; i<N; i++)` per determinar si es pot paral·lelitzar directament o si cal aplicar transformacions prèvies.

A) Dependències sobre la matriu resultat `C1[][]` (Nucli 1).

- **Dependències detectades:**
  - S0 (δ, δ^o, $\delta^/$) S0: Dins del bucle intern sobre k, múltiples elements $AD[k]$ amb el mateix `AD[k].i` acumulen a la mateixa posició `C1[AD[k].i][i]` mitjançant l'operador `+=` (lectura+escriptura). Això genera dependències de flux, de sortida i anti-dependències transportades pel bucle intern.
- **Anàlisi:** Respecte al bucle exterior sobre i, cada iteració escriu a una **columna diferent** de $C1$ (la columna i-èsima). Per tant, no hi ha conflictes d'escriptura entre iteracions d'i: les iteracions són completament independents entre si. Les dependències internes del bucle sobre k queden encapsulades dins de cada iteració. Les estructures `AD[]` i `B[][]` són de només lectura.

B) Dependències sobre el vector temporal `VBcol[]` (Nucli 2).

- **Dependències detectades:**
  - S1 (δ·0) S2: La fase d'expansió escriu a `VBcol` i la fase de càlcul llegeix d'aquest vector (flux, distància 0).
  - S1 (δ^o·0) S3: L'expansió escriu a `VBcol` i la neteja també hi escriu (sortida, distància 0).
  - S2 ($\delta^/$·0) S3: El càlcul llegeix `VBcol` abans que la neteja l'esborri (anti-dependència, distància 0).
  - S3 (δ^o·1) S1: La neteja de la iteració i i l'expansió de la iteració $i+1$ escriuen ambdues a `VBcol` (sortida, distància 1).
  - S3 (δ·1) S2: La neteja de la iteració i posa a 0 les posicions que el càlcul de la iteració $i+1$ llegirà (flux, distància 1).
  - S3 (δ^o·1) S3: La neteja d'una iteració sobreescriu les mateixes posicions que la neteja de la iteració anterior (sortida, distància 1).
- **Anàlisi:** El vector `VBcol` actua com a buffer temporal compartit que segueix un cicle d'expansió→ús→neteja dins de cada iteració. La dependència de flux de distància 1 entre S3 i S1 genera un cicle al graf de dependències del bucle exterior, cosa que en principi impediria la paral·lelització directa per iteracions

d'i. No obstant això, si cada fil disposa de la seva pròpia còpia privada de `VBcol`, el cicle es trenca completament, ja que cada iteració opera de forma autònoma sobre el seu buffer local.

C) Dependències sobre la matriu resultat `C2[][]` (Nucli 2).

- **Dependències detectades:**
  - S2 (δ, δ^o, $\delta^/$) S2: Dins del bucle intern sobre k, múltiples elements $AD[k]$ amb el mateix `AD[k].i` acumulen a la mateixa posició `C2[AD[k].i][i]` mitjançant l'operador `+=`. Genera dependències de flux, de sortida i anti-dependències transportades pel bucle intern.
- **Anàlisi:** Respecte al bucle exterior sobre i, cada iteració escriu a una columna diferent de $C2$ (la columna i-èsima). Per tant, no hi ha conflictes d'escriptura entre iteracions d'i sobre $C2$: l'accés és un patró d'independència total respecte al bucle exterior. Les dependències internes del bucle sobre k queden encapsulades dins de cada iteració.

D) Dependències sobre les estructures de lectura `AD[]`, `BD[]`, `jBD[]`, `B[][]`.

- **Anàlisi:** Aquestes estructures són de només lectura dins dels tres nuclis computacionals. No generen cap dependència d'escriptura i, per tant, poden ser compartides de forma segura entre tots els fils sense cap risc de condició de carrera.

Conclusió: El Nucli 1 és directament paral·lelitzable amb `#pragma omp for`, ja que les seves iteracions són completament independents entre si. Al Nucli 2, l'única dependència que impedeix la paral·lelització directa del bucle exterior és el cicle generat per `VBcol`. Amb la privatització d'aquest vector, el bucle es converteix en un conjunt d'iteracions totalment independents sobre les columnes de $C2$.

2.2 Nucli 3: MD $\times$ MD $\rightarrow$ MD (Resultat Dispers)

2.2.1 Codi seqüencial

El tercer nucli és estructuralment similar al Nucli 2, però afegeix una fase de compressió que emmagatzema el resultat en format dispers (`CD[]`) mitjançant un comptador global `neleC` i un vector temporal addicional `VCcol`.

Listing 3: Nucli 3: MD $\times$ MD $\rightarrow$ MD

```
// Inicialitzacio
    neleC = 0;
    for (j=0; j<N; j++)
        VBcol[j] = VCcol[j] = 0;

// Bucle principal (per columnes)
    for(i=0; i<N; i++)
    {
        // Fase 1: Expansio de la columna i de B
        for (k=jBD[i]; k<jBD[i+1]; k++)
S1:         VBcol[BD[k].i] = BD[k].v;

        // Fase 2: Calcul de la columna i de C
        for (k=0; k<ND; k++)
S2:         VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j];

        // Fase 3: Neteja i compressio
        for (j=0; j<N; j++)
        {
S3:         VBcol[j] = 0;
            if (VCcol[j])
            {
S4:             CD[neleC].i = j;
S5:             CD[neleC].j = i;
S6:             CD[neleC].v = VCcol[j];
S7:             VCcol[j] = 0;
S8:             neleC++;
            }
        }
    }
```

2.2.2 Graf de Dependències del Nucli 3

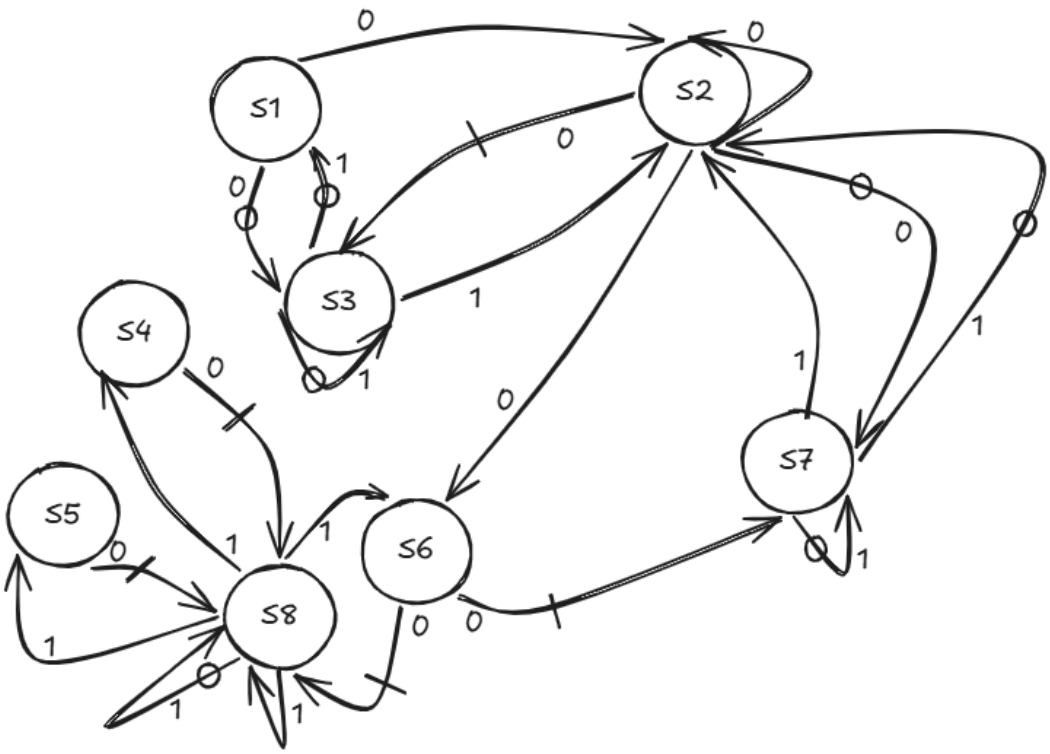

Figura 2: Graf de dependències del Nucli 3 (MD × MD → MD)

2.2.3 Llistat i tipus de dependències del Nucli 3

El Nucli 3 hereta l'estructura del Nucli 2 (expansió→ús→neteja de `VBcol`) i n'afegeix de noves associades a `VCcol`, `CD[]` i `neleC`. S'analitzen les dependències del bucle exterior `for(i=0; i<N; i++)`.

A) Dependències sobre el vector temporal `VBcol[]` (idèntiques al Nucli 2).

- **Dependències detectades:**
  - S1 (δ·0) S2: L'expansió escriu a `VBcol` i el càlcul el llegeix (flux, distància 0).
  - S1 (δ^o·0) S3: L'expansió i la neteja escriuen ambdues a `VBcol` (sortida, distància 0).
  - S2 ($\delta^/$·0) S3: El càlcul llegeix `VBcol` abans que la neteja l'esborri (anti-dependència, distància 0).
  - S3 (δ^o·1) S1: La neteja de la iteració i i l'expansió de la iteració $i+1$ escriuen ambdues a `VBcol` (sortida, distància 1).
  - S3 (δ·1) S2: La neteja de la iteració i posa a 0 les posicions que el càlcul de la iteració $i+1$ llegirà (flux, distància 1).
  - S3 (δ^o·1) S3: La neteja d'una iteració sobreescriu les mateixes posicions que la neteja de la iteració anterior (sortida, distància 1).
- **Anàlisi:** El patró és idèntic al del Nucli 2: un cicle d'expansió→ús→neteja que genera dependències de distància 1 formant un cicle al graf. Requereix privatització de `VBcol` per trencar-lo.

B) Dependències sobre el vector temporal `VCcol[]`.

- **Dependències detectades:**
  - S2 (δ, δ^o·0) S2: Dins del bucle intern sobre k, múltiples elements $AD[k]$ amb el mateix `AD[k].i` acumulen a la mateixa posició de `VCcol` (flux i sortida, distància 0).
  - S2 (δ·0) S4: El valor acumulat a `VCcol[j]` es llegeix a la comprovació `if(VCcol[j])` (flux, distància 0).

- S2 (δ·0) S6: El valor acumulat a `VCcol[j]` es llegeix per emmagatzemar-lo a `CD` (flux, distància 0).
- S2 (δ^o·0) S7: L'acumulació i la neteja escriuen ambdues a `VCcol` (sortida, distància 0).
- S6 ($\delta^/$·0) S7: La lectura de `VCcol[j]` per a la compressió es fa abans que la neteja l'esborri (anti-dependència, distància 0).
- S7 (δ·1) S2: La neteja de la iteració i posa a 0 les posicions que l'acumulació de la iteració $i+1$ llegirà amb el `+=` (flux, distància 1).
- S7 (δ^o·1) S2: La neteja i l'acumulació de la iteració següent escriuen ambdues a `VCcol` (sortida, distància 1).
- S7 (δ^o·1) S7: La neteja d'una iteració sobreescriu les mateixes posicions que la neteja anterior (sortida, distància 1).

- **Anàlisi:** `VCcol` segueix un patró d'acumulació→lectura→neteja anàleg al de `VBcol`. Les dependències de distància 1 generen un cicle al graf que impedeix la paral·lelització directa. Requereix privatització per trencar-lo, de manera que cada fil disposi de la seva pròpia còpia del vector d'acumulació.

C) Dependències sobre el comptador `neleC`.

- **Dependències detectades:**
  - S4 ($\delta^/$·0) S8: `neleC` es llegeix com a índex a S4 i posteriorment s'escriu a S8 (anti-dependència, distància 0).
  - S5 ($\delta^/$·0) S8: Ídem per a S5 (anti-dependència, distància 0).
  - S6 ($\delta^/$·0) S8: Ídem per a S6 (anti-dependència, distància 0).
  - S8 (δ·1) S4: L'increment de `neleC` a la iteració anterior determina l'índex de S4 a la següent (flux, distància 1).
  - S8 (δ·1) S5: Ídem per a S5 (flux, distància 1).
  - S8 (δ·1) S6: Ídem per a S6 (flux, distància 1).
  - S8 (δ, δ^o·1) S8: L'increment es llegeix i s'escriu entre iteracions consecutives (flux i sortida, distància 1).
- **Anàlisi:** El comptador `neleC` actua com un índex seqüencial d'inserció al vector comprimit `CD`. El seu valor a cada iteració depèn de quants elements no nuls s'han trobat a totes les iteracions anteriors, generant un cicle de dependències que impedeix la paral·lelització directa. Aquesta és la dependència més problemàtica del Nucli 3: cada fil necessitaria saber exactament quantes entrades han produït els fils anteriors per escriure a la posició correcta de `CD`. La solució requerirà una estratègia específica com la compressió local per fil amb fusió posterior o un càlcul previ d'offsets.

D) Dependències sobre l'array comprimit `CD[]`.

- **Anàlisi:** Les sentències S4, S5 i S6 escriuen a camps diferents del mateix element (`.i`, `.j`, `.v`) i, com que `neleC` sempre creix, cada escriptura va a una posició única de l'array. Per tant, `CD[]` no genera dependències conflictives per si sol. El problema real no és l'array en si, sinó el comptador `neleC` que en determina la posició d'escriptura.

Conclusió: El Nucli 3 hereta la necessitat de privatitzar `VBcol` (com al Nucli 2) i afegeix la privatització de `VCcol`. No obstant això, presenta un repte addicional que el Nucli 2 no tenia: la compressió seqüencial controlada per `neleC`. L'estratègia de paral·lelització haurà de separar el càlcul (paral·lelitzable amb privatització) de la compressió (que requereix un tractament especial per mantenir la correcció de `CD`).

3 Optimitzacions Aplicades

En aquesta secció s'analitzen les transformacions realitzades sobre el codi seqüencial de referència per obtenir la versió paral·lelitzada amb OpenMP. Cada subsecció presenta el fragment original (*antic*) i el nou (*optimitzat*), i n'explica el raonament tècnic.

3.1 Inicialització paral·lela amb *first-touch*

Versió seqüencial

El codi original inicialitzava totes les matrius de forma seqüencial mitjançant `bzero`:

Listing 4: Inicialització seqüencial

```
bzero(C,  sizeof(int)*(N*N));
bzero(C1, sizeof(int)*(N*N));
bzero(C2, sizeof(int)*(N*N));
```

Versió optimitzada

S'ha substituït per un bucle paral·lel que inicialitza fila a fila amb `memset`:

Listing 5: Inicialització paral·lela amb first-touch

```
#pragma omp parallel for schedule(static)
for (int idx = 0; idx < N; idx++) {
    memset(A[idx],  0, sizeof(int) * N);
    memset(B[idx],  0, sizeof(int) * N);
    memset(C[idx],  0, sizeof(int) * N);
    memset(C1[idx], 0, sizeof(int) * N);
    memset(C2[idx], 0, sizeof(int) * N);
}
```

Justificació

En arquitectures **NUMA** (*Non-Uniform Memory Access*), la primera escriptura a una pàgina de memòria (*first touch*) determina en quin node físic s'ubicarà. Si el fil principal fa `bzero` de forma seqüencial, tota la matriu queda assignada al node del fil principal i la resta de fils l'han d'accedir de forma remota, augmentant la latència d'accés. Distribuint la inicialització amb `omp for schedule(static)`, cada fil "toca" les files que posteriorment calcularà, forçant el sistema operatiu a col·locar cada pàgina al node de memòria correcte.

3.2 Introducció del paràmetre BLOCK_SIZE i blocs de càlcul

Versió seqüencial

El codi original recorria les columnes d'una en una, sense cap agrupació:

Listing 6: Recorregut per columnes individuals (seqüencial)

```
for (i = 0; i < N; i++) {
    for (k = jBD[i]; k < jBD[i+1]; k++)
        VBcol[BD[k].i] = BD[k].v;
    for (k = 0; k < ND; k++)
        C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];
    for (j = 0; j < N; j++)
        VBcol[j] = 0;
}
```

Versió optimitzada

S'ha introduït una constant `BLOCK_SIZE = 16` i el recorregut s'agrupa en blocs de columnes. Per a cada bloc, s'utilitzen buffers privats bidimensionals `priv_VBcol[N][BLOCK_SIZE]` i `priv_VCcol[N][BLOCK_SIZE]` que mantenen les dades del bloc actiu a la memòria cau L1:

Listing 7: Recorregut per blocs de columnes (optimitzat)

```
#define BLOCK_SIZE 16

int (*priv_VBcol)[BLOCK_SIZE] = malloc(sizeof(int) * N * BLOCK_SIZE);
int (*priv_VCcol)[BLOCK_SIZE] = malloc(sizeof(int) * N * BLOCK_SIZE);

for (int ib = 0; ib < N; ib += BLOCK_SIZE) {
    memset(priv_VBcol, 0, sizeof(int) * N * BLOCK_SIZE);
    for (int i = ib; i < ib + BLOCK_SIZE; i++)
        for (int k = jBD[i]; k < jBD[i+1]; k++)
            priv_VBcol[BD[k].i][i - ib] = BD[k].v;
    for (int k = 0; k < ND; k++) {
        int rA = AD[k].i, cA = AD[k].j, vA = AD[k].v;
        for (int i = ib; i < ib + BLOCK_SIZE; i++)
            C2[rA][i] += vA * priv_VBcol[cA][i - ib];
    }
}
```

Justificació

La tècnica de *cache tiling* o *blocking* consisteix a subdividir el domini de treball en blocs que càpiguen a la memòria cau L1. Amb `BLOCK_SIZE = 16` columnes simultànies, el buffer `priv_VBcol` ocupa $8000 \times 16 \times 4 \approx 512\,\text{KB}$, ajustat a la mida típica de la memòria cau L2. D'aquesta manera, un cop carregat el bloc de columnes de B, tots els accessos al vector d'expansió es produeixen des de cau, reduint dràsticament els *cache misses* en comparació amb la versió que exposa i neteja el vector columna a columna.

A més, el layout bidimensional `[fila][offset_dins_bloc]` garanteix que els accessos contigus dins del bucle intern recorren posicions de memòria consecutives, afavorint la vectorització automàtica pel compilador.

3.3 Fase 1: Matriu dispersa × Matriu densa → Densa (C1)

Versió seqüencial

Listing 8: MD × M seqüencial

```
for (i = 0; i < N; i++)
    for (k = 0; k < ND; k++)
        C1[AD[k].i][i] += AD[k].v * B[AD[k].j][i];
```

Versió optimitzada

Listing 9: MD × M paral·lelitzat amb blocs

```
#pragma omp for schedule(static) nowait
for (int ib = 0; ib < N; ib += BLOCK_SIZE) {
    for (int k = 0; k < ND; k++) {
        int rA = AD[k].i, cA = AD[k].j, vA = AD[k].v;
        for (int i = ib; i < ib + BLOCK_SIZE; i++)
            C1[rA][i] += vA * B[cA][i];
    }
}
```

Justificació

El codi original fixava la columna de sortida al bucle extern, de manera que l'accés a `C1` i a `B` saltava de fila en fila produint *stride-N* i una elevada taxa de *cache misses*. La nova versió agrupa el treball en blocs de columnes: per a cadascun, la fila de `B` i la fila de `C1` es mantenen calentes a la cau mentre s'itera sobre `BLOCK_SIZE` posicions contigues. S'utilitza `schedule(static)` perquè totes les iteracions tenen un cost pràcticament idèntic (es processen sempre tots els `ND` elements d'A), i la clàusula `nowait` permet al fil continuar sense esperar els companys abans d'abordar la fase següent.

3.4 Fase 2: Matriu dispersa × Matriu dispersa → Densa (C2)

Versió seqüencial

Listing 10: MD × MD → M seqüencial

```
for (i = 0; i < N; i++) {
    for (k = jBD[i]; k < jBD[i+1]; k++)
        VBcol[BD[k].i] = BD[k].v;
    for (k = 0; k < ND; k++)
        C2[AD[k].i][i] += AD[k].v * VBcol[AD[k].j];
    for (j = 0; j < N; j++)
        VBcol[j] = 0;
}
```

Versió optimitzada

Listing 11: MD × MD → M paral·lelitzat amb buffer privat

```
#pragma omp for schedule(dynamic, 32) nowait
for (int ib = 0; ib < N; ib += BLOCK_SIZE) {
    memset(priv_VBcol, 0, sizeof(int) * N * BLOCK_SIZE);
    for (int i = ib; i < ib + BLOCK_SIZE; i++)
        for (int k = jBD[i]; k < jBD[i+1]; k++)
            priv_VBcol[BD[k].i][i - ib] = BD[k].v;
    for (int k = 0; k < ND; k++) {
        int rA = AD[k].i, cA = AD[k].j, vA = AD[k].v;
        for (int i = ib; i < ib + BLOCK_SIZE; i++)
            C2[rA][i] += vA * priv_VBcol[cA][i - ib];
    }
}
```

Justificació

La versió seqüencial compartia un únic vector auxiliar `VBcol` i requeria un bucle de neteja de mida N per cada columna processada. En la versió paral·lela, cada fil disposa del seu propi `priv_VBcol` (reservat amb `malloc` dins del bloc `omp parallel`), la qual cosa elimina qualsevol condició de carrera i evita el *false sharing* que es produiria si dos fils accedissin a posicions properes d'un vector compartit. S'utilitza `schedule(dynamic, 32)` perquè la densitat de no-nuls per columna de B és irregular: amb un repartiment dinàmic de blocs de 32 columnes, els fils que finalitzen aviat reben nous blocs i es manté l'ocupació dels nuclis fins al final.

3.5 Fase 3: Matriu dispersa × Matriu dispersa → Dispersa (CD)

Versió seqüencial

Listing 12: MD × MD → MD seqüencial

```
neleC = 0;
for (i = 0; i < N; i++) {
    for (k = jBD[i]; k < jBD[i+1]; k++)
        VBcol[BD[k].i] = BD[k].v;
    for (k = 0; k < ND; k++)
        VCcol[AD[k].i] += AD[k].v * VBcol[AD[k].j];
    for (j = 0; j < N; j++) {
        VBcol[j] = 0;
        if (VCcol[j]) {
            CD[neleC].i = j; CD[neleC].j = i;
            CD[neleC].v = VCcol[j];
            VCcol[j] = 0; neleC++;
        }
    }
}
```

Versió optimitzada

Listing 13: MD × MD → MD paral·lelitzat amb atomic capture

```
#pragma omp for schedule(dynamic, 8)
for (int ib = 0; ib < N; ib += BLOCK_SIZE) {
    memset(priv_VBcol, 0, sizeof(int) * N * BLOCK_SIZE);
    memset(priv_VCcol, 0, sizeof(int) * N * BLOCK_SIZE);

    for (int i = ib; i < ib + BLOCK_SIZE; i++)
        for (int k = jBD[i]; k < jBD[i+1]; k++)
            priv_VBcol[BD[k].i][i - ib] = BD[k].v;

    for (int k = 0; k < ND; k++) {
        int rA = AD[k].i, cA = AD[k].j, vA = AD[k].v;
        for (int i = ib; i < ib + BLOCK_SIZE; i++)
            priv_VCcol[rA][i - ib] += vA * priv_VBcol[cA][i - ib];
    }

    for (int i = ib; i < ib + BLOCK_SIZE; i++) {
        for (int j = 0; j < N; j++) {
            if (priv_VCcol[j][i - ib]) {
                int pos;
                #pragma omp atomic capture
                pos = neleC++;
                CD[pos].i = j; CD[pos].j = i;
                CD[pos].v = priv_VCcol[j][i - ib];
            }
        }
    }
}
```

Justificació

Aquesta és la fase més complexa de paral·lelitzar perquè el resultat és un array de mida dinàmica (CD) sobre el qual múltiples fils han d'escriure simultàniament. La versió seqüencial mantenia un únic comptador global `neleC`, però en un entorn paral·lel qualsevol accés concurrent a aquest comptador introduiria condicions de carrera.

La solució adoptada combina dos mecanismes:

1. **Buffers privats per fil.** Tant `priv_VBcol` com `priv_VCcol` es reserven dins del bloc `parallel`, de manera que cada fil disposa de les seves pròpies estructures d'acumulació. Això elimina els conflictes en l'escriptura dels resultats parcials i evita el *false sharing* entre fils adjacents.

2. **`atomic capture` per a la inserció.** Quan un fil detecta un element no nul al seu `priv_VCcol`, reserva de forma atòmica una posició al vector global `CD` incrementant `neleC`. La semàntica *capture* garanteix que la lectura del valor anterior i l'increment es produeixen com una operació indivisible, de manera que dues posicions mai es solaparan. Malgrat que l'operació atòmica introdueix una certa serialització, el cost es manté baix perquè el nombre d'elements no nuls de C és molt inferior a N^2, limitant la contesa sobre el comptador.

S'utilitza `schedule(dynamic, 8)` amb blocs petits de 8 iteracions perquè la densitat de no-nuls varia molt entre blocs de columnes: un repartiment dinàmic fi permet que els fils que acaben ràpidament absorbeixin nous blocs sense deixar nuclis ociosos.

4 Avaluació del Rendiment

Per avaluar l'impacte de la paral·lelització s'han realitzat bateries de proves en dues arquitectures de memòria compartida: la màquina **Teen** i la màquina **Orca**. S'ha sol·licitat accés exclusiu als nodes (`--exclusive`) per evitar interferències d'altres usuaris i s'han registrat els temps reals d'execució (*elapsed*). L'acceleració o *Speedup* (S) s'ha calculat com $S = T_s/T_p$, on T_s és el temps de la versió seqüencial compilada amb `-O3` i T_p el de la versió paral·lela.

4.1 Avaluació a la màquina Orca

La topologia d'Orca es basa en un processador **AMD Ryzen Threadripper PRO 3995WX** (microarquitectura Zen 2) amb **64 nuclis físics** i **128 fils lògics** via SMT. El temps seqüencial base ha estat $T_s = \mathbf{48.59\,s}$.

Nombre de fils	**Temps d'execució (s)**	**Speedup** ($S = 48.59/T_p$)
1 (Seqüencial Base)	48.59	1.00x
2	4.99	9.74x
4	2.95	16.47x
8	2.13	22.81x
16	2.27	21.41x
32	2.23	21.79x
64	2.32	20.94x
128	2.60	18.69x

Taula 1: Resultats d'escalabilitat — Màquina Orca (64 Cores / 128 Threads)

La mitjana harmònica dels *speedups* (de 2 a 128 fils) és:

$$\overline{S}_{\text{harm}} = \frac{7}{\frac{1}{9.74} + \frac{1}{16.47} + \frac{1}{22.81} + \frac{1}{21.41} + \frac{1}{21.79} + \frac{1}{20.94} + \frac{1}{18.69}} \approx \mathbf{17.45x}$$

El comportament és clarament *memory-bound*: l'escalabilitat s'estanca a partir dels **8 fils** (22.81x) i decreix progressivament fins als 128 fils (18.69x). Quan tots els canals de memòria es troben saturats, afegir nous fils no incrementa el rendiment sinó que introdueix contenció al bus de memòria i als nivells de memòria cau compartits (LLC), resultant en un *speedup* negatiu a partir d'aquell punt. L'overhead d'OpenMP (creació de fils, barreres implícites i l'operació `atomic capture` sobre `neleC`) també s'amplifica amb l'augment del nombre de fils, contribuint a la degradació observada.

4.2 Avaluació a la màquina Teen

La màquina Teen disposa d'un processador **AMD Ryzen 7 7700** (microarquitectura Zen 4) amb **8 nuclis físics** i **16 fils lògics** via SMT. El temps seqüencial base ha estat $T_s = \mathbf{16.26\,s}$. S'han avaluat les execucions escalant des d'1 fins a 16 fils, aprofitant la totalitat dels nuclis lògics disponibles.

Nombre de fils	**Temps d'execució (s)**	**Speedup** ($S = 16.26/T_p$)
1 (Seqüencial Base)	16.26	1.00x
2	3.62	4.49x
4	2.08	7.82x
8	1.49	10.91x
16	1.35	12.04x

Taula 2: Resultats d'escalabilitat — Màquina Teen (8 Cores / 16 Threads)

La mitjana harmònica dels *speedups* (de 2 a 16 fils) és:

$$\overline{S}_{\text{harm}} = \frac{4}{\frac{1}{4.49} + \frac{1}{7.82} + \frac{1}{10.91} + \frac{1}{12.04}} \approx \mathbf{7.61x}$$

4.3 Comparativa entre arquitectures

Malgrat que Orca ofereix 8 vegades més nuclis físics que Teen, els resultats en temps absolut mostren una realitat contraintuïtiva: **Teen aconsegueix temps d'execució inferiors** emprant únicament 16 fils davant del millor cas d'Orca amb només 8 fils.

Màquina	T_s **(s)**	**Millor** T_p **(s)**	**Millor Speedup**
Orca (128 fils)	48.59	2.13 (8 fils)	22.81x
Teen (16 fils)	16.26	1.35 (16 fils)	12.04x

Taula 3: Comparativa del millor temps absolut obtingut a cada màquina

Les causes d'aquest avantatge de Teen s'articulen al voltant de tres factors principals:

1. **Microarquitectura superior (Zen 4 vs Zen 2).** El Ryzen 7 7700 de Teen implementa la microarquitectura Zen 4, amb un IPC (*Instructions Per Cycle*) significativament superior al Zen 2 d'Orca, memòries cau L1/L2 més grans per nucli i freqüències de rellotge més elevades. Això explica per si sol que el temps seqüencial de Teen (16.26s) sigui gairebé **3 vegades més ràpid** que el d'Orca (48.59s), malgrat executar exactament el mateix codi.

2. **Memòria DDR5 amb ample de banda superior per nucli.** Teen utilitza memòria DDR5, que ofereix una latència menor i un ample de banda superior al DDR4 d'Orca. En un problema *memory-bound* com la multiplicació de matrius disperses, on el coll d'ampolla és la velocitat d'accés als arrays, un major ample de banda de memòria es tradueix directament en temps d'execució inferiors.

3. **Saturació de la memòria d'Orca a partir de 8 fils.** Com es pot observar a la Taula 1, el *speedup* d'Orca assoleix el màxim a 8 fils i decreix a partir d'aleshores. Amb 64 nuclis Zen 2 competint simultàniament pels mateixos canals de memòria, el bus queda saturat ràpidament. En canvi, Teen aconsegueix escalar el seu rendiment fins als 16 fils (el seu màxim lògic) arribant al seu temps mínim d'1.35s.

5 Conclusions

En aquesta pràctica s'ha dut a terme la paral·lelització d'un codi de multiplicació de matrius disperses, seguint una metodologia rigorosa d'anàlisi, transformació i avaluació de rendiment mitjançant OpenMP.

L'anàlisi dels grafs de dependències ha resultat crític per garantir la correcció del codi. Al Nucli 1, l'absència de dependències entre iteracions del bucle exterior ha permès una paral·lelització directa. Als Nuclis 2 i 3, el repte principal requeia en els vectors temporals `VBcol` i `VCcol`, que generaven cicles de dependències de distància 1. La **privatització** d'aquests vectors, declarant còpies locals dins de la regió paral·lela, ha estat la transformació clau per trencar els cicles i habilitar la paral·lelització.

Pel que fa al Nucli 3, el comptador seqüencial `neleC` ha suposat un repte addicional. L'ús d'una estratègia de càlcul paral·lel combinada amb una compressió seqüencial ha demostrat ser eficient. Segons la Llei d'Amdahl, donat que la compressió representa una fracció ínfima del temps total, mantenir-la seqüencial no penalitza l'escalabilitat global. A més, encapsular els tres nuclis en una única regió `#pragma omp parallel` ha permès eliminar l'overhead de creació i destrucció repetida del *pool* de fils.

L'avaluació empírica ha confirmat clarament la naturalesa ***memory-bound*** d'aquest problema. A diferència d'altres algorismes centrats en còmput, aquí el coll d'ampolla resideix en l'ample de banda de la memòria i els *cache misses* derivats dels accessos indirectes als arrays dispersos. Això s'ha evidenciat a la màquina Orca, on l'speedup màxim assolit ha estat de **22.81x amb 8 fils**, patint una degradació de rendiment en afegir-ne més a causa de la contenció al bus de memòria. D'altra banda, la màquina Teen ha demostrat que una **arquitectura més moderna (Zen 4) amb memòria més ràpida (DDR5)** pot superar àmpliament en temps absolut un sistema amb molts més nuclis però tecnologia anterior, assolint el millor temps global de la pràctica d'**1.35 segons a 16 fils**. L'eficiència per nucli i l'ample de banda han resultat ser els factors decisius per maximitzar el rendiment.